

ARRAYS & TWO POINTERS

DSA. PART - 1/8



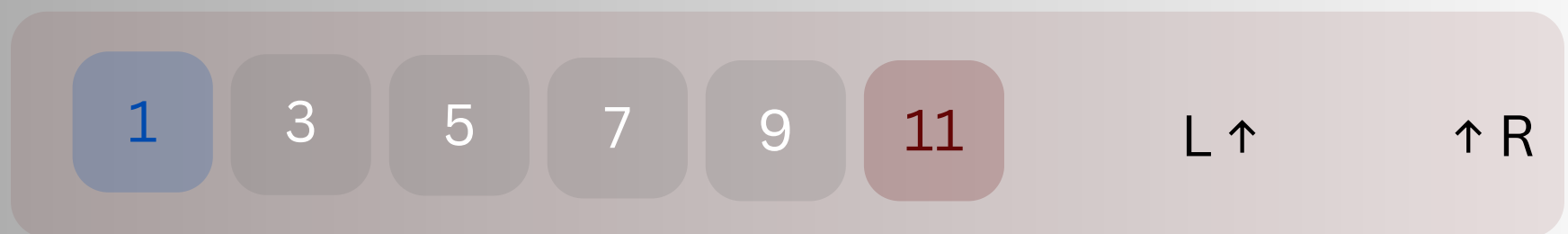
@java_in.depth

WHAT IS TWO POINTER?

The core idea explained simply

The Idea

Instead of using nested loops ($O(n^2)$), we place two pointers at different positions in the array and move them smartly — reducing time to $O(n)$.



When to use it?

- Sorted array
- Pair/triplet sum
- Remove duplicates
- Palindrome
- Sliding window

4 TWO-POINTER PATTERNS

Know these → solve 80% of array questions

Opposite Ends

L at start, R at end. Move toward center.

Two Sum, Palindrome

→→

Same Direction

Slow & fast pointer moving forward.

Remove Dups, Cycle

Sliding Window

Window of size k slides across array.

Max Sum Subarray

Anchor + Runner

One fixed, one explores ahead.

Dutch Flag, Partition

TWO SUM (SORTED ARRAY)

Given a sorted array, find two numbers that add up to a target. Return their indices.

```
int[] twoSum(int[] nums, int target) {  
    int L = 0, R = nums.length - 1;  
    while (L < R) {  
        int sum = nums[L] + nums[R];  
        if (sum == target)  
            return new int[]{L, R};  
        else if (sum < target) L++;  
        else R--;  
    }  
    return new int[]{-1, -1};  
}
```

Key Insight:

Sum too small? Move L right. Sum too big?
Move R left. $O(n)$ time, $O(1)$ space.

MAXIMUM SUBARRAY SUM

Find the contiguous subarray with the largest sum. (Kadane's Algorithm)

```
int maxSubArray(int[] nums) {  
    int maxSum = nums[0];  
    int curr = nums[0];  
    for (int i=1; i<nums.length; i++) {  
        // extend or restart window  
        curr = Math.max(nums[i], curr+nums[i]);  
        maxSum = Math.max(maxSum, curr);  
    }  
    return maxSum;  
}
```

Key Insight:

If current sum goes negative → restart from current element. Always track global max.

$O(n)$ time, $O(1)$ space.

CONTAINER WITH MOST WATER

Given heights of lines, find two lines that form a container holding the most water.

```
int maxArea(int[] height) {  
    int L=0, R=height.length-1, max=0;  
    while (L < R) {  
        int h = Math.min(height[L], height[R]);  
        max = Math.max(max, h*(R-L));  
        // move the shorter line inward  
        if(height[L] < height[R]) L++;  
        else R--;  
    }  
    return max;  
}
```

Key Insight:

Area = $\min(\text{height}) \times \text{width}$. Always move the shorter pointer — moving the taller one can never increase area!

5 TIPS TO ACE ARRAYS

Say these out loud in your interview

- Always ask if the array is sorted — it changes the entire approach.
- Brute force first, then optimize. Never jump to $O(n)$ without explaining $O(n^2)$ first.
- Edge cases: empty array, single element, all duplicates, negative numbers.
- Name your variables clearly — "L is left pointer, R is right" — interviewers love this.
- Trace your code on the given example before saying "I'm done".

TIME & SPACE COMPLEXITY

Memorize these — interviewers always ask

Problem:	Time:	Space:	Pattern:
Two Sum (sorted)	$O(n)$	$O(1)$	Opposite ends
Remove Duplicates	$O(n)$	$O(1)$	Slow / Fast
Max Subarray	$O(n)$	$O(1)$	Kadane's
Container Water	$O(n)$	$O(1)$	Opposite ends
3Sum	$O(n^2)$	$O(1)$	Fix + 2ptr
Merge Sorted Arrs	$O(n^2)$	$O(n+m)$	Two arrays

Remember

Two Pointer almost always gives $O(n)$ time with $O(1)$ space. That's the magic!

MUST-SOLVE PROBLEMS

LeetCode list before your interview

#1 Two Sum — HashMap vs Two Pointer approach

#15 3Sum — Fix one element + two pointer on rest

#42 Trapping Rain Water — Classic two pointer hard

#75 Sort Colors — Dutch National Flag (3 pointers!)

#283 Move Zeroes — Slow/fast pointer in-place

DON'T MAKE THESE MISTAKES

These cost candidates the offer

Off-by-one errors — Always double-check $L < R$ vs $L \leq R$ in while conditions.

Missing edge cases — What if array is empty?
Length = 1? All same elements?

Moving wrong pointer — In sorted arrays, think logically before choosing which side to advance.

Not using sorted property — If sorted, two pointer or binary search is almost always optimal.

WHAT 'S NEXT?

Your comment decides the next post!

01 Arrays & Two Pointers Done

02 Strings & Sliding Window

03 Linked Lists

04 Trees & BST

05 Dynamic Programming

06 Graphs (BFS / DFS)

07 Sorting & Searching

08 Stack & Queue

Comment Your number